🐳 AWS Containers & Serverless — Category 5: Complete Interview Guide

> **Target Audience:** DevOps / SRE / Platform / Cloud Engineers
> **Difficulty:** Beginner → Advanced
> **Topics Covered:** ECS, ECR, ECS Launch Types, EKS, Lambda, VPC CNI, IRSA, Karpenter, Cold Starts, SnapStart, App Mesh, Fargate

---

📋 Table of Contents

1. 5.1 ECS — Clusters, Tasks, Services, Task Definitions
2. 5.2 ECR — Image Lifecycle, Scanning, Replication
3. 5.3 ECS Launch Types — EC2 vs Fargate
4. 5.4 EKS — Architecture, Node Groups, Managed vs Self-Managed
5. 5.5 Lambda — Execution Model, Triggers, Layers, Concurrency
6. 5.6 EKS — Networking (VPC CNI), IRSA, Cluster Autoscaler vs Karpenter
7. 5.7 Lambda — Cold Starts, Provisioned Concurrency, SnapStart
8. 5.8 App Mesh & Service Mesh Concepts
9. 5.9 Fargate — Resource Limits, Networking, Security

5.1 ECS — Clusters, Tasks, Services, Task Definitions

🟢 What It Is in Simple Terms

ECS (Elastic Container Service) is AWS's native container orchestrator. You give it Docker containers, it runs them and keeps them healthy. Think of it as Kubernetes but simpler and fully managed by AWS — no control plane to maintain.

🔍 Why It Exists / What Problem It Solves

Running containers on raw EC2 means you manually schedule containers, handle failures, manage scaling, wire up load balancers, and deal with rolling deployments yourself. ECS automates all of that — it's a container scheduler that knows where to place containers, restarts failed ones, integrates natively with AWS services, and handles blue/green deployments.

⚙️ How It Works Internally

```
ECS Architecture — Key Building Blocks:

┌──────────────────────────────────────────────────────────────────────────────┐
```

```
│                            ECS CLUSTER                                │
│  (logical grouping of compute resources)                              │
│                                                                       │
│  ┌──────────────────────────────────────────────────────────────┐    │
│  │                      ECS SERVICE                             │    │
│  │  (keeps N tasks running, integrates with ALB)              │    │
│  │                                                              │    │
│  │   Task 1          Task 2          Task 3                    │    │
│  │  ┌─────────┐      ┌─────────┐      ┌─────────┐              │    │
│  │  │Container│      │Container│      │Container│              │    │
│  │  │(Docker) │      │(Docker) │      │(Docker) │              │    │
│  │  └─────────┘      └─────────┘      └─────────┘              │    │
│  │   ↑ defined by Task Definition                               │    │
│  └──────────────────────────────────────────────────────────────┘    │
│                                                                       │
│  Compute: EC2 instances (Container Instances) OR Fargate              │
└──────────────────────────────────────────────────────────────────────┘

Request Flow:
Internet → ALB → Target Group → ECS Service → Running Tasks
```

🧩 Key Components Deep Dive

Clusters

```
ECS Cluster:
├── Logical grouping of:
│   - EC2 instances (Container Instances running ECS agent)
│   - Fargate capacity
│   - Or both (mixed)
├── One cluster per environment (dev/staging/prod)
├── Contains multiple Services
└── Can span multiple AZs

ECS Agent:
├── Docker container running on every EC2 instance
├── Communicates with ECS control plane
├── Reports: available CPU/memory, running tasks
└── Receives: instructions to start/stop tasks
```

Task Definitions

```
Task Definition = blueprint for running containers
(Like a Dockerfile but for an entire pod/task)

Key fields:
{
```

```
  "family": "web-app",         // name of task definition
  "revision": 3,               // auto-increments on each update
  "networkMode": "awsvpc",     // awsvpc | bridge | host | none
  "requiresCompatibilities": ["FARGATE"],
  "cpu": "512",                // task-level vCPU (256=0.25, 512=0.5,
1024=1)
  "memory": "1024",            // task-level memory in MiB

  "containerDefinitions": [{
    "name": "web",
    "image": "123456.dkr.ecr.us-east-1.amazonaws.com/web:v1.2.3",
    "portMappings": [{"containerPort": 8080, "protocol": "tcp"}],
    "cpu": 256,
    "memory": 512,
    "memoryReservation": 256,  // soft limit
    "essential": true,         // if this dies, kill whole task
    "environment": [
      {"name": "ENV", "value": "prod"}
    ],
    "secrets": [{
      "name": "DB_PASSWORD",
      "valueFrom": "arn:aws:secretsmanager:...:secret:db-pass"
    }],
    "logConfiguration": {
      "logDriver": "awslogs",
      "options": {
        "awslogs-group": "/ecs/web-app",
        "awslogs-region": "us-east-1",
        "awslogs-stream-prefix": "ecs"
      }
    },
    "healthCheck": {
      "command": ["CMD-SHELL", "curl -f http://localhost:8080/health ||
exit 1"],
      "interval": 30,
      "timeout": 5,
      "retries": 3
    },
    "mountPoints": [{
      "sourceVolume": "efs-data",
      "containerPath": "/data"
    }]
  }],

  "volumes": [{
    "name": "efs-data",
    "efsVolumeConfiguration": {
      "fileSystemId": "fs-0abc123",
      "transitEncryption": "ENABLED"
    }
  }],

  "taskRoleArn": "arn:aws:iam::123:role/ecs-task-role",
```

```
  "executionRoleArn": "arn:aws:iam::123:role/ecs-execution-role"
}
```

Two IAM Roles in ECS — Critical Interview Topic:

```
┌──────────────────────────────────────────────────────────┐
│ Task Role (taskRoleArn)                                  │
│ = what your APPLICATION can do                           │
│ Examples: read S3, write DynamoDB, call SQS              │
│ Used by: your container's code                           │
├──────────────────────────────────────────────────────────┤
│ Execution Role (executionRoleArn)                        │
│ = what ECS AGENT can do on your behalf                   │
│ Examples: pull image from ECR, write logs to CloudWatch, │
│           read secrets from Secrets Manager              │
│ Used by: ECS infrastructure, not your app code           │
└──────────────────────────────────────────────────────────┘

⚠ Missing execution role = can't pull image from ECR
⚠ Missing task role = app gets AccessDenied calling AWS APIs
```

Tasks

```
Task = a running instance of a Task Definition
     = one or more containers running together

Task lifecycle:
PROVISIONING → PENDING → RUNNING → STOPPING → STOPPED

Common container exit codes: Exit 0 = clean shutdown. Exit 1 = application
error (check logs). Exit 137 = OOM kill (increase memory limit). Exit 139
= segfault. Exit code is visible in ECS console under stopped task
details.

Key task concepts:
├── Tasks are ephemeral — they start, run, stop
├── ECS does NOT restart stopped tasks (the Service does)
├── Task = one or more tightly coupled containers (like a pod)
└── All containers in a task share:
    - Network namespace (awsvpc mode: shared ENI)
    - EFS mounts
    - Task metadata
```

```
# Running a one-off task (batch job, migration, etc.)
aws ecs run-task \
  --cluster prod-cluster \
```

```
  --task-definition db-migration:5 \
  --launch-type FARGATE \
  --network-configuration '{
    "awsvpcConfiguration": {
      "subnets": ["subnet-abc123"],
      "securityGroups": ["sg-xyz789"],
      "assignPublicIp": "DISABLED"
    }
  }'
```

Services

```
ECS Service = long-running task manager + load balancer integration

Service capabilities:
├── Maintain desired count (restarts failed tasks)
├── Rolling deployments (replace old tasks with new)
├── Blue/green deployments (via CodeDeploy)
├── Integration with ALB / NLB
├── Auto Scaling (via Application Auto Scaling)
└── Service discovery (via AWS Cloud Map)
```

```
# Create ECS service
aws ecs create-service \
  --cluster prod-cluster \
  --service-name web-service \
  --task-definition web-app:3 \
  --desired-count 6 \
  --launch-type FARGATE \
  --deployment-configuration '{
    "maximumPercent": 200,
    "minimumHealthyPercent": 100
  }' \
  --load-balancers '[{
    "targetGroupArn": "arn:aws:elasticloadbalancing:...",
    "containerName": "web",
    "containerPort": 8080
  }]' \
  --network-configuration '{
    "awsvpcConfiguration": {
      "subnets": ["subnet-a", "subnet-b", "subnet-c"],
      "securityGroups": ["sg-web"],
      "assignPublicIp": "DISABLED"
    }
  }'
```

```
Deployment types:
1. Rolling Update (default):
   Replace old tasks gradually.
   minimumHealthyPercent + maximumPercent control the speed.
   ⚠ Rollback = redeploy previous task definition manually.

2. Blue/Green (via CodeDeploy):
   Old = Blue (still serving traffic)
   New = Green (deployed, tested, then traffic shifted)
   Instant rollback (just shift traffic back).
   Supports: canary, linear, all-at-once shifting.

3. External (your own deployment controller):
   For custom deployment logic.

The deployment circuit breaker must be explicitly enabled — it is off by
default. Without it, ECS will loop indefinitely launching and terminating
failing tasks, burning cost and time. Always enable with rollback: true
for production services.
```

```bash
# Service Auto Scaling — register target
aws application-autoscaling register-scalable-target \
  --service-namespace ecs \
  --resource-id service/prod-cluster/web-service \
  --scalable-dimension ecs:service:DesiredCount \
  --min-capacity 2 \
  --max-capacity 20

# Target tracking on CPU utilization
aws application-autoscaling put-scaling-policy \
  --policy-name cpu-tracking \
  --service-namespace ecs \
  --resource-id service/prod-cluster/web-service \
  --scalable-dimension ecs:service:DesiredCount \
  --policy-type TargetTrackingScaling \
  --target-tracking-scaling-policy-configuration '{
    "PredefinedMetricSpecification": {
      "PredefinedMetricType": "ECSServiceAverageCPUUtilization"
    },
    "TargetValue": 60.0
  }'
```

💬 Short Crisp Interview Answer

"ECS is AWS's container orchestrator. The key building blocks are: Task Definitions (blueprints defining containers, CPU/memory, IAM roles, logging), Tasks (running instances of a definition), Services (long-running managers that maintain desired task count, integrate with ALBs, and handle rolling deployments), and Clusters (logical groups of compute capacity). There are two critical IAM roles — the execution role lets

ECS pull images from ECR and write logs, while the task role is what your application code uses to call AWS services. The most common mistake is confusing these two: missing execution role means ECS can't start your container; missing task role means your app can't call AWS APIs."

🏭 Real World Production Example

```
Production microservices on ECS:

Cluster: prod-cluster (Fargate)

Services:
├── api-service:       desired=6, min=2, max=20
│   Scaling: CPU > 60% → scale out
│   Deploy: Blue/Green via CodeDeploy (canary 10% → 100%)
│   ALB: api.myapp.com → target group → port 8080
│
├── worker-service:    desired=3, min=1, max=50
│   Scaling: SQS queue depth > 100 → scale out
│   No ALB (pulls from SQS queue)
│
└── scheduler-service: desired=1, min=1, max=1
    Runs cron jobs (EventBridge → ECS run-task)

Task Definition (api):
- Image: ECR repo, pinned to digest (not :latest)
- CPU: 512, Memory: 1024
- Secrets: from Secrets Manager via execution role
- Logs: CloudWatch /ecs/api-service
- Health check: /health endpoint

Networking:
- awsvpc mode (each task gets its own ENI)
- Private subnets only
- Security group: allow 8080 from ALB SG only
```

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
Task Role vs Execution Role	The most common ECS interview mistake. Know both cold
essential: true	If essential container stops, whole task stops. Set correctly
:latest tag	Never use in production. Image won't update on redeploy unless ECS re-pulls. Pin to digest
memoryReservation vs memory	Reservation = soft limit (can burst). memory = hard limit (OOM killed)

Gotcha	Detail
Task definition is immutable	Each update creates a new revision. Old revisions remain
Service desired count	If you manually stop a task, Service restarts it automatically
deregistration delay	Set ALB target group deregistration delay to 30s for fast blue/green rollbacks

- `CannotPullContainerError` **has multiple root causes** — always check in this order: execution role permissions → VPC Endpoints (if private subnet) → security group outbound 443 → cross-account ECR policy → correct region in image URI.
- **ECR requires THREE endpoints in private subnets** — ecr.api, ecr.dkr, AND the S3 Gateway Endpoint. Missing the S3 endpoint causes layer pull failures even when the other two are present, because ECR stores image layers in S3.

5.2 ECR — Image Lifecycle, Scanning, Replication

🟢 What It Is in Simple Terms

ECR (Elastic Container Registry) is AWS's managed Docker image registry — like Docker Hub but private, integrated with IAM, and living inside your AWS account. You push container images to ECR, ECS/EKS pulls them at runtime.

⚙️ How It Works Internally

```
Docker image workflow with ECR:

Developer                 ECR                     ECS/EKS
────────                  ───                     ───────
docker build              Repository              Task/Pod
docker tag    ──push──►   123456.dkr.ecr.         ──pull──►  Running
docker push               us-east-1.amazonaws               container
                          .com/my-app:v1.2.3

ECR repository types:
├── Private: default, IAM-controlled
└── Public:  public.ecr.aws/... — publicly accessible
    (for sharing open-source images, avoids Docker Hub rate limits)
```

```
# Authenticate Docker to ECR
aws ecr get-login-password --region us-east-1 | \
```

```
  docker login --username AWS --password-stdin \
  123456.dkr.ecr.us-east-1.amazonaws.com
```

🧩 Image Lifecycle Policies

```
Purpose: automatically clean up old/unused images.
Without lifecycle policy → images accumulate forever → storage costs
mount.

Lifecycle policy rules:
├── Rule priority: lower number = higher priority
├── Actions: expire (delete) images
└── Selection criteria:
    - tagStatus: tagged | untagged | any
    - tagPrefixList: ["v1.", "dev-"]
    - countType: imageCountMoreThan | sinceImagePushed
    - countNumber: 10 (keep last 10)
    - countUnit: days (for sinceImagePushed)

Example policy:
{
  "rules": [
    {
      "rulePriority": 1,
      "description": "Keep last 5 production images",
      "selection": {
        "tagStatus": "tagged",
        "tagPrefixList": ["v"],
        "countType": "imageCountMoreThan",
        "countNumber": 5
      },
      "action": {"type": "expire"}
    },
    {
      "rulePriority": 2,
      "description": "Delete untagged images after 1 day",
      "selection": {
        "tagStatus": "untagged",
        "countType": "sinceImagePushed",
        "countUnit": "days",
        "countNumber": 1
      },
      "action": {"type": "expire"}
    },
    {
      "rulePriority": 3,
      "description": "Delete dev images older than 14 days",
      "selection": {
        "tagStatus": "tagged",
        "tagPrefixList": ["dev-", "pr-"],
```

```
        "countType": "sinceImagePushed",
        "countUnit": "days",
        "countNumber": 14
      },
      "action": {"type": "expire"}
    }
  ]
}
```

```
# Enable tag immutability on production repos
aws ecr put-image-tag-mutability \
  --repository-name my-app \
  --image-tag-mutability IMMUTABLE

# ⚠ Without IMMUTABLE: CI can accidentally overwrite v1.2.3
#    IMMUTABLE prevents re-pushing an existing tag
```

🧩 Image Scanning

```
ECR offers two scanning modes:

1. Basic Scanning (free):
   ├── On push or manual trigger
   ├── Uses Clair open-source scanner
   └── Detects OS-level CVEs only

2. Enhanced Scanning (paid, via AWS Inspector):
   ├── Continuous scanning (rescans as new CVEs discovered)
   ├── Scans OS packages AND application packages
   │   (node_modules, pip packages, Ruby gems, etc.)
   ├── Results in AWS Inspector console + EventBridge events
   └── Integration with Security Hub
```

```
# Enable scan on push (basic)
aws ecr put-image-scanning-configuration \
  --repository-name my-app \
  --image-scanning-configuration scanOnPush=true

# Retrieve scan results
aws ecr describe-image-scan-findings \
  --repository-name my-app \
  --image-id imageTag=v1.2.3

# CI/CD pipeline gate — fail build on CRITICAL CVEs
CRITICAL=$(aws ecr describe-image-scan-findings \
  --repository-name my-app \
```

```
  --image-id imageTag=$IMAGE_TAG \
  --query 'imageScanFindings.findingSeverityCounts.CRITICAL' \
  --output text)

if [ "$CRITICAL" -gt "0" ]; then
  echo "CRITICAL CVEs found. Failing build."
  exit 1
fi
```

🧩 Replication

```
Two types of ECR replication:

1. Cross-Region Replication:
   Automatically replicate images to other regions.
   Use: deploy same image from local ECR in each region
        (faster pull, lower cross-region data transfer cost)

   Multi-Region EKS — ECR Replication is MANDATORY:
   Without replication:
    ├── ap-southeast-1 nodes pull images from us-east-1 ECR on every pod
start
    ├── Adds pull latency (cross-region ~100-200ms per layer)
    ├── Creates hard dependency on us-east-1 availability
    │   → if us-east-1 ECR is degraded, ap-southeast-1 pods cannot start
    └── Defeats the entire purpose of multi-region deployment
   With replication:
    ├── Images available locally in each region's ECR
    ├── Pod starts are fast and region-independent
    └── DR scenario works correctly — ap-southeast-1 is truly self-
sufficient

2. Cross-Account Replication:
   Replicate to other AWS accounts.
   Use: central image registry account → push to prod/dev accounts
```

```
# Configure replication
aws ecr put-replication-configuration \
  --replication-configuration '{
    "rules": [{
      "destinations": [
        {"region": "us-west-2", "registryId": "123456789012"},
        {"region": "eu-west-1", "registryId": "999888777666"}
      ],
      "repositoryFilters": [{
        "filter": "prod-",
        "filterType": "PREFIX_MATCH"
      }]
```

```
    }]
  }'
```

```
⚠ Replication is eventual — not instant.
   Allow a few minutes for image to appear in destination.
   Don't deploy to destination region immediately after source push.

Pull-through cache:
├── ECR can proxy and cache Docker Hub / Quay / GCR images
├── On first pull → fetches from upstream, caches in ECR
├── Subsequent pulls → served from ECR (no Docker Hub rate limit)
└── Use: avoid Docker Hub pull rate limits in CI/CD
```

💬 Short Crisp Interview Answer

"ECR is AWS's managed container image registry with IAM-based access control. Key operational concerns are: lifecycle policies to expire old/untagged images automatically to control storage costs — without them images accumulate indefinitely; image scanning — basic scanning detects OS CVEs on push, enhanced scanning via AWS Inspector also catches application-level vulnerabilities continuously; and tag immutability, which prevents overwriting a production image tag. ECR also supports cross-region and cross-account replication so ECS/EKS clusters can pull from a local ECR endpoint rather than crossing regions, reducing latency and data transfer costs."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
No lifecycle policy = infinite storage cost	Old images never cleaned up. Always add lifecycle policy
Tag mutability	Without IMMUTABLE, anyone can overwrite v1.2.3 — a CI bug can corrupt a production image
Replication is eventual	Don't deploy immediately after push if cross-region replication is required
ECR auth token expiry	Token from `get-login-password` expires in 12 hours. ECS handles this automatically; CI must refresh
Cross-account pull	Destination account needs a resource policy on the ECR repo allowing the pull

---

5.3 ECS Launch Types — EC2 vs Fargate

🟢 What It Is in Simple Terms

ECS can run containers on two types of compute: EC2 instances you manage yourself (EC2 launch type), or AWS-managed compute where you never see a server (Fargate launch type). Same ECS API, same task definitions — just different underlying compute.

⚙️ Deep Comparison

```
┌──────────────────────────┬──────────────────────────────┬──────────────────────
──────┐
│ Feature                  │ EC2 Launch Type              │ Fargate Launch Type
│
├──────────────────────────┼──────────────────────────────┼──────────────────────
──────┤
│ Server management        │ You manage EC2 instances     │ AWS manages
everything    │
│ OS patching              │ Your responsibility          │ AWS's responsibility
│
│ Capacity planning        │ You size & scale cluster     │ No capacity planning
│
│ Container density        │ Multiple per EC2 (pack)      │ 1 task = isolated
env     │
│ Cost model               │ Pay for EC2 (even idle)      │ Pay per vCPU/sec +
GB/s   │
│ GPU support              │ ✅ p3, p4, g4 instances      │ ❌ No GPU support
│
│ Windows containers       │ ✅ Windows ECS AMI           │ ✅ Windows Fargate
│
│ SSH/exec access          │ ✅ SSH to EC2                │ ✅ ECS Exec only
│
│ Custom AMI               │ ✅                           │ ❌
│
│ Spot support             │ ✅ Spot EC2 in cluster       │ ✅ Fargate Spot
│
│ Max CPU per task         │ Entire EC2 instance          │ 16 vCPU
│
│ Max memory per task      │ Entire EC2 instance          │ 120 GB
│
│ Network mode             │ bridge, awsvpc, host         │ awsvpc only
│
│ EFS support              │ ✅                           │ ✅
│
│ EBS support              │ ✅                           │ ✅ ephemeral volume
only   │
│ Startup time             │ Faster (prewarmed EC2)       │ ~30s cold start
│
└──────────────────────────┴──────────────────────────────┴──────────────────────
──────┘
```

🧩 EC2 Launch Type — Deep Dive

```
How it works:
├── You provision EC2 instances into your ECS cluster
├── ECS agent on each EC2 registers capacity with cluster
├── ECS scheduler places tasks on EC2 instances
└── You control: instance type, scaling, AMI, storage

Container Instance (EC2 in ECS cluster):
├── Must run ECS-optimized AMI (or install ECS agent manually)
├── ECS agent communicates with ECS control plane
├── Reports available CPU/memory for placement decisions
└── Instance profile needs: ecs:RegisterContainerInstance, etc.

Placement strategies (EC2 only):
├── binpack:  pack containers onto fewest EC2s (minimize cost)
├── spread:   distribute across AZs and instances (HA)
└── random:   random placement

Placement constraints:
├── memberOf: expression-based (e.g., attribute:ecs.instance-type =~ m5.*)
└── distinctInstance: each task on a different EC2 instance

⚠ Over-provisioned EC2 capacity costs money even with no tasks.
   Under-provisioned = tasks fail to place (PENDING state forever).
   Use Capacity Providers to auto-scale the EC2 cluster itself.

Capacity Providers (EC2 mode with auto-scaling):
├── Attach an ASG to ECS cluster as a Capacity Provider
├── ECS signals ASG to scale out when tasks can't be placed
├── ECS signals ASG to scale in when tasks are stopped
├── MANAGED_SCALING: ECS drives ASG scaling automatically
└── MANAGED_TERMINATION_PROTECTION: prevents EC2 with running
    tasks from being terminated during scale-in
```

🧩 Fargate Launch Type — Deep Dive

```
How it works:
├── You specify CPU and memory in Task Definition
├── Submit task/service to ECS
├── AWS provisions isolated compute behind the scenes
├── Task runs — you never see the underlying server
└── Billing: per vCPU-second + per GB-second (while task runs)

Fargate pricing (approximate):
vCPU:   $0.04048 per vCPU-hour
Memory: $0.004445 per GB-hour

Example: 1 vCPU, 2GB, running 24/7:
```

```
Daily cost: (1 × $0.04048 + 2 × $0.004445) × 24 = ~$1.18/day
Monthly: ~$35/month per task

Fargate networking:
├── awsvpc ONLY — each task gets its own ENI
├── Each task has its own Security Group
├── Each task has its own private IP
└── NO bridge or host networking
```

```
# Fargate Spot — up to 70% cheaper
aws ecs create-service \
  --cluster prod-cluster \
  --service-name worker-service \
  --capacity-provider-strategy '[
    {"capacityProvider": "FARGATE",      "weight": 1, "base": 1},
    {"capacityProvider": "FARGATE_SPOT", "weight": 3}
  ]'
# 1 guaranteed Fargate task (base) + 75% Spot thereafter

# ECS Exec — interactive shell inside Fargate task (no SSH)
aws ecs execute-command \
  --cluster prod-cluster \
  --task abc123 \
  --container web \
  --interactive \
  --command "/bin/bash"
# Uses SSM Session Manager under the hood
# Must enable: enableExecuteCommand=true on service
```

🧩 When to Choose Which

```
Use Fargate when:
├── You want zero server management
├── Variable/unpredictable workloads (scale to zero)
├── Small team / startup
├── You're okay with slightly higher per-unit cost
└── Strong security isolation per task is important

Use EC2 when:
├── GPU workloads (ML training, inference)
├── Windows containers (complex)
├── Extreme density (pack many small containers cheaply)
├── Custom AMI with specific kernel modules
├── Steady, predictable high-volume workloads
│   (EC2 Savings Plans make EC2 much cheaper)
└── Compliance requires control over underlying host

Hybrid pattern (Capacity Provider strategy):
```

```
Use FARGATE for base capacity + FARGATE_SPOT for burst.
Or: EC2 Spot instances (via Capacity Provider) + Fargate fallback.
```

💬 Short Crisp Interview Answer

"ECS has two launch types. EC2 launch type runs containers on EC2 instances you provision and manage — you control instance types, placement strategies, and get features like GPU support and custom AMIs, but you're responsible for OS patching and capacity planning. Fargate is serverless — you specify CPU and memory in the task definition and AWS manages the rest — no EC2 instances to manage, stronger per-task isolation, awsvpc networking only. Fargate costs more per compute unit but eliminates operational overhead. For production, I'd use Fargate for most services and EC2 with Capacity Providers for GPU workloads or cases where you're packing many small containers and cost matters a lot. Fargate Spot can cut Fargate costs by 70% for batch or non-critical workloads."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
Fargate cold start	~30 seconds to start a new task. EC2 is faster with pre-warmed instances
Fargate max resources	16 vCPU, 120 GB per task. Anything larger needs EC2
EC2 task placement failures	If no EC2 has enough CPU/memory → task stays PENDING
Fargate awsvpc only	Cannot use bridge networking. Each task needs a Security Group and ENI
ENI limits per instance	awsvpc mode on EC2 uses ENIs — large clusters hit ENI limits. Fargate avoids this
ECS Exec requirement	Must set enableExecuteCommand=true at service creation — can't add retroactively

---

5.4 EKS — Architecture, Node Groups, Managed vs Self-Managed

🟢 What It Is in Simple Terms

EKS (Elastic Kubernetes Service) is AWS's managed Kubernetes service. AWS runs the Kubernetes control plane (etcd, API server, scheduler, controller manager) — you manage the worker nodes. It gives you full Kubernetes with AWS integrations.

🔍 Why EKS over ECS?

```
ECS:
├── Simpler API, native AWS
├── Tightly integrated with AWS services
├── No Kubernetes knowledge needed
├── Less flexible / portable
└── Best for: teams going all-in on AWS

EKS:
├── Standard Kubernetes API (portable workloads)
├── Larger ecosystem (Helm, ArgoCD, Prometheus, etc.)
├── More control (custom networking, storage plugins)
├── Can run same manifests on-prem or other clouds
└── Best for: existing Kubernetes workloads, large teams,
             organizations with Kubernetes expertise

Choose ECS when: AWS-native, simplicity first
Choose EKS when: Kubernetes expertise, portability, ecosystem
```

⚙️ EKS Architecture

```
EKS Architecture:

┌──────────────────────────────────────────────────────────────┐
│                    AWS MANAGED CONTROL PLANE                 │
│                    (you pay, AWS manages)                    │
│                                                              │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────────────┐  │
│  │ API Server │  │   etcd     │  │ Scheduler +          │  │
│  │ (kubectl)  │  │  (cluster  │  │ Controller Manager   │  │
│  │            │  │   state)   │  │                      │  │
│  └──────────────┘  └──────────────┘  └──────────────────────┘  │
│                                                              │
│  Multi-AZ, 3 replicas, auto-patched, 99.95% SLA              │
└──────────────────────────────────────────────────────────────┘
                       │ Kubernetes API
                       ▼
┌──────────────────────────────────────────────────────────────┐
│                    YOUR VPC (you manage)                     │
│                                                              │
│  Node Group AZ-a   Node Group AZ-b   Node Group AZ-c         │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐       │
│  │ EC2 Worker │  │ EC2 Worker │  │ EC2 Worker │       │
│  │  ┌─────────┐ │  │  ┌─────────┐ │  │  ┌─────────┐ │       │
│  │  │   Pod   │ │  │  │   Pod   │ │  │  │   Pod   │ │       │
│  │  │   Pod   │ │  │  │   Pod   │ │  │  │   Pod   │ │       │
│  │  └─────────┘ │  │  └─────────┘ │  │  └─────────┘ │       │
│  └──────────────┘  └──────────────┘  └──────────────┘       │
│                                                              │
│  Or: Fargate Profiles (serverless nodes)                     │
```

```
└──────────────────────────────────────────────────────────────────────────┘

Cost:
├── Control Plane: $0.10/hr = $72/month flat (always)
└── Worker nodes: EC2 or Fargate (normal pricing)
```

🧩 Node Groups

Managed Node Groups (MNG)

```
What AWS manages:
├── EC2 instance provisioning
├── Node AMI (EKS-optimized) — auto-updates available
├── Instance lifecycle (graceful drain on scale-in)
├── ASG creation and management
└── Node registration with cluster

What you manage:
├── Instance type selection
├── Min/max/desired node count
├── Node group IAM role
└── Launch template customization (optional)
```

```
# Create managed node group
aws eks create-nodegroup \
  --cluster-name prod-cluster \
  --nodegroup-name app-nodes \
  --instance-types m5.xlarge \
  --ami-type AL2_x86_64 \
  --capacity-type ON_DEMAND \
  --scaling-config minSize=2,maxSize=10,desiredSize=3 \
  --subnets subnet-a subnet-b subnet-c \
  --node-role arn:aws:iam::123:role/eks-node-role \
  --labels role=app
```

```
Benefits of MNG:
├── Graceful node drain on scale-in (pods migrate before termination)
├── Automatic AMI updates available (one-click upgrade)
└── Spot support: CAPACITY_TYPE=SPOT for 70-80% savings
```

Self-Managed Nodes

```
You fully manage the EC2 instances:
├── Create your own ASG
├── Use EKS-optimized AMI or build custom
├── Bootstrap script runs at startup:
    /etc/eks/bootstrap.sh prod-cluster
├── Register node with cluster manually
└── Handle node draining yourself on scale-in

Use cases:
├── Need custom AMI (kernel modules, proprietary software)
├── ARM/Graviton instances not yet in MNG
├── Windows worker nodes
└── Very specific instance types or Spot configurations

⚠ Self-managed nodes require significantly more operational work.
   Use Managed Node Groups unless you have a specific reason not to.
```

Fargate Profiles (Serverless Nodes)

```
No EC2 worker nodes at all.
EKS schedules pods on AWS-managed Fargate compute.

Fargate Profile selects WHICH pods run on Fargate:
├── Namespace selector: run all pods in "serverless" namespace
└── Label selector: run pods with label environment=staging
```

```
aws eks create-fargate-profile \
  --cluster-name prod-cluster \
  --fargate-profile-name app-profile \
  --pod-execution-role-arn arn:aws:iam::123:role/fargate-pod-role \
  --subnets subnet-a subnet-b \
  --selectors '[
    {"namespace": "production", "labels": {"type": "api"}},
    {"namespace": "staging"}
  ]'
```

```
⚠ EKS Fargate limitations:
├── No DaemonSets (no per-node agents — use sidecar instead)
├── No privileged containers
├── No GPU
├── No EBS volumes (EFS only)
├── Slower startup than EC2 nodes
└── Limited size (4 vCPU, 30GB per pod max)
```

🧩 Key EKS Concepts

```
# Create EKS add-on (e.g., VPC CNI)
aws eks create-addon \
  --cluster-name prod-cluster \
  --addon-name vpc-cni \
  --addon-version v1.14.1-eksbuild.1 \
  --service-account-role-arn arn:aws:iam::123:role/vpc-cni-role

# Configure kubectl access
aws eks update-kubeconfig --name prod-cluster --region us-east-1
```

```yaml
# aws-auth ConfigMap: maps IAM identities to Kubernetes RBAC
apiVersion: v1
kind: ConfigMap
metadata:
  name: aws-auth
  namespace: kube-system
data:
  mapRoles: |
    - rolearn: arn:aws:iam::123:role/eks-node-role
      username: system:node:{{EC2PrivateDNSName}}
      groups:
        - system:bootstrappers
        - system:nodes
    - rolearn: arn:aws:iam::123:role/developer-role
      username: developer
      groups:
        - dev-team
```

💬 Short Crisp Interview Answer

"EKS is AWS's managed Kubernetes service. AWS runs the control plane — API server, etcd, scheduler — across multiple AZs with a 99.95% SLA. You manage the worker nodes. There are three node types: Managed Node Groups where AWS handles EC2 provisioning, ASG, and graceful draining; Self-Managed Nodes for custom AMIs or specialized hardware; and Fargate Profiles for serverless pods with no EC2 to manage. I'd use Managed Node Groups for most workloads — they give you AMI updates, graceful draining, and Spot support without the overhead of self-managed nodes. The key EKS operational concerns are: IRSA for pod-level IAM, the aws-auth ConfigMap for access control, and choosing between Cluster Autoscaler and Karpenter for node scaling."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail

Gotcha	Detail
Control plane cost	$0.10/hr always, even with zero nodes. Don't create EKS clusters for quick tests
aws-auth misconfiguration	Lock yourself out by removing your role from aws-auth. Always have break-glass access
Fargate DaemonSets	Not supported. Log forwarders, monitoring agents must be sidecars
Node IAM role vs pod IAM (IRSA)	Node IAM role is for EC2 infrastructure. Pod IAM is IRSA. Never put app permissions on node role
EKS API version skew	kubectl must be within 1 minor version of cluster version
Managed Node Group drain	When scaling in, MNG drains pods first — but only if PodDisruptionBudgets allow it

---

5.5 Lambda — Execution Model, Triggers, Layers, Concurrency

🟢 What It Is in Simple Terms

Lambda is AWS's serverless compute. You upload code, define triggers, and AWS runs it — you never provision servers, never manage scaling, and pay only when your code is actually executing. The unit of work is a function that responds to events.

🔍 Why It Exists / What Problem It Solves

For event-driven workloads (process an S3 upload, respond to an API call, react to a DynamoDB change), you don't want a server sitting idle waiting for events. Lambda gives you compute that scales from zero to thousands of parallel invocations and back to zero automatically.

```
⚠ Lambda VPC Decision Rule:

Put Lambda in VPC ONLY IF it needs to reach private resources
(RDS, ElastiCache, EC2 in private subnet).

If Lambda only calls public AWS services (S3, DynamoDB,
Secrets Manager, SNS, SQS):
→ Keep Lambda OUTSIDE VPC — simpler, cheaper, no NAT needed

If Lambda MUST be in VPC:
→ Add S3/DynamoDB Gateway Endpoints (free)
→ Add Interface Endpoints for Secrets Manager, SSM etc.
→ Never rely on NAT Gateway for AWS service calls
```

⚙️ Execution Model Internals

```
Lambda Execution Environment:

1. First invocation (cold start):
   ├── AWS allocates a microVM (Firecracker)
   ├── Downloads your deployment package / container image
   ├── Starts the language runtime (Node, Python, Java, etc.)
   ├── Runs your initialization code (outside handler)
   └── Runs your handler function

2. Subsequent invocations (warm):
   ├── AWS REUSES the same execution environment
   ├── Skips bootstrap steps above
   ├── Runs your handler function directly
   └── /tmp is STILL POPULATED from previous invocation!

3. Execution environment recycled when:
   ├── Lambda scales down (idle for ~15 min)
   ├── Deployment update (new version deployed)
   └── Lambda service maintenance

Memory configuration:
├── 128MB to 10,240MB (10GB)
├── CPU scales linearly with memory
│   (1.8GB = 1 vCPU, 3.6GB = 2 vCPU)
├── More memory = more CPU = faster execution
└── Use AWS Lambda Power Tuning tool to find optimal setting
```

🧩 Triggers (Event Sources)

```
Lambda is invoked three ways:

1. SYNCHRONOUS (caller waits for response):
   ├── API Gateway / ALB  → HTTP requests
   ├── Lambda URL         → direct HTTPS endpoint
   ├── Cognito            → pre/post signup triggers
   └── CloudFront Lambda@Edge

2. ASYNCHRONOUS (fire and forget):
   ├── S3 events          → object created, deleted, etc.
   ├── SNS                → push notifications
   ├── EventBridge        → scheduled rules, custom events
   └── SES                → email receiving

   Async behavior:
   ├── Lambda retries automatically (2 retries on failure)
```

```
    ├── Events queued up to 6 hours if throttled
    ├── Dead Letter Queue (DLQ): failed events → SQS/SNS
    └── Event Destination: route success/failure to
SNS/SQS/EventBridge/Lambda

3. POLLING (Lambda polls the source):
    ├── SQS               → Lambda polls queue, batches messages
    ├── DynamoDB Streams  → process change events
    ├── Kinesis Streams   → real-time stream processing
    └── MSK / Kafka       → consume Kafka topics
```

```
Key SQS trigger configuration:
├── Batch size: 1-10,000 messages per invocation
├── BatchWindow: 0-300 seconds (wait to fill batch)
├── MaxConcurrency: limit parallel Lambda invocations
├── FunctionResponseTypes: "ReportBatchItemFailures"
│   (partially succeed — only retry FAILED messages in batch)
└── ⚠ SQS VisibilityTimeout must be 6× Lambda timeout

Kinesis/DynamoDB Streams:
├── One Lambda invocation per shard
├── BisectOnFunctionError: split failing batch into halves
│   (find the poison-pill message)
└── DestinationConfig: send failures to SQS/SNS after retries
```

🧩 Lambda Layers

```
What layers are:
Reusable packages/libraries deployed separately from function code.
Instead of bundling 500MB of dependencies in every package,
put them in a Layer and reference it.

Layer directory structure by runtime:
Python:  python/lib/python3.11/site-packages/
Node.js: nodejs/node_modules/
Java:    java/lib/
Generic: bin/, lib/

Benefits:
├── Smaller deployment packages (faster deploy)
├── Share common libraries across functions
├── Update dependency once → affects all referencing functions
├── Separate deployment of code vs dependencies
└── AWS provides official layers (Pandas, etc.)

Limits:
├── Up to 5 layers per function
```

```
├── Total unzipped size (function + all layers): 250MB
└── Layer can be shared across accounts
```

```
# Build and publish a layer
mkdir -p layer/python
pip install requests -t layer/python/
zip -r layer.zip layer/

aws lambda publish-layer-version \
  --layer-name requests-layer \
  --zip-file fileb://layer.zip \
  --compatible-runtimes python3.11 python3.10

# Attach layer to function
aws lambda update-function-configuration \
  --function-name my-function \
  --layers arn:aws:lambda:us-east-1:123:layer:requests-layer:3
```

```
⚠ Layers are version-pinned.
   Updating a layer version does NOT automatically update functions.
   You must update each function to reference the new layer version.
```

🧩 Concurrency

```
Concurrency = number of function instances running simultaneously

Types:

1. Unreserved Concurrency:
   ├── Default pool — all functions draw from it freely
   ├── Account limit: 1,000 concurrent executions (soft limit)
   └── Can request increase via support ticket

2. Reserved Concurrency:
   ├── Allocate a dedicated pool to a specific function
   ├── OTHER functions cannot use this pool (guarantee)
   ├── ALSO acts as a maximum cap (throttle ceiling)
   └── Setting reserved=0 → completely throttle a function

3. Provisioned Concurrency:
   ├── Pre-initialize execution environments
   ├── Eliminates cold starts for pre-warmed instances
   └── Costs: $0.015/provisioned-GBs-hour

Concurrency formula:
Concurrent executions = requests_per_second × avg_duration_in_seconds
```

```
Example: 1,000 req/s × 0.1s avg  = 100 concurrent executions
         1,000 req/s × 2.0s avg  = 2,000 concurrent executions!
         (slow functions consume much more concurrency)

Throttle behavior:
├── Synchronous: 429 TooManyRequests error returned to caller
└── Async: event queued or dropped (with DLQ/destination)

⚠ One noisy Lambda function can exhaust account concurrency pool
   and starve other functions. Use Reserved Concurrency to isolate.
```

💬 Short Crisp Interview Answer

"Lambda is event-driven serverless compute. The execution model is: each invocation runs in an isolated microVM — on cold start, AWS bootstraps the environment and runs your init code; on warm invocations, the same environment is reused. Triggers fall into three categories: synchronous (API Gateway, ALB), asynchronous (S3, SNS, EventBridge — with automatic retries and DLQ support), and polling (SQS, Kinesis, DynamoDB Streams — Lambda polls and batches). Layers package shared dependencies separately from function code. Concurrency is the key operational concept — unreserved is a shared pool, reserved guarantees and caps a function, and provisioned pre-warms environments to eliminate cold starts."

🏭 Real World Production Example

```
E-commerce order processing pipeline:

API Gateway (sync) → Lambda (order-api)
  → validates order, writes to DynamoDB, publishes to SNS

SNS fan-out → Lambda (inventory-service) [async]
           → Lambda (payment-service)    [async]
           → Lambda (email-service)      [async]

SQS Queue → Lambda (fulfillment) [polling]
  - Batch size: 10
  - Visibility timeout: 60s (lambda timeout: 10s × 6)
  - ReportBatchItemFailures: enabled
  - DLQ: fulfillment-dlq (after 3 failures)

S3 (invoice bucket) → Lambda (invoice-generator) [async]
  → On s3:ObjectCreated → generate PDF → upload to output bucket

Concurrency configuration:
order-api:          Reserved=500, Provisioned=50 (zero cold start)
fulfillment:        Reserved=100 (protect from noisy neighbors)
invoice-generator:  Unreserved (bursty, tolerates cold starts)
```

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
/tmp persists across warm invocations	If env is reused, /tmp has data from previous invocation. Never assume clean /tmp
SQS visibility timeout	Must be 6× Lambda timeout. Otherwise message becomes visible again while still processing → duplicate
Concurrency = per account	One function at 900 concurrent = only 100 left for all other functions
Async retry = up to 3 runs	2 automatic retries on failure = function may run 3 times total. Must be idempotent
Layers not auto-updated	Must update function to reference new layer version after updating layer
Kinesis shard blocking	If Lambda fails on a batch, it retries indefinitely (blocking that shard). Use BisectOnFunctionError

- **SQS visibility timeout misconfiguration is the #1 cause of duplicate Lambda processing** — always set visibility timeout to minimum 6× Lambda timeout. This is a mandatory configuration check for any SQS-triggered Lambda in production.
- **Idempotency is not optional for SQS consumers** — SQS Standard at-least-once delivery means duplicates can occur even with correct visibility timeout. Always design Lambda handlers to be idempotent using a unique message identifier as the idempotency key.
- **ReportBatchItemFailures prevents the 9-succeed-1-fail reprocessing problem** — without it, a single failed message in a batch causes all successfully processed messages to return to the queue. Always enable for production SQS-triggered Lambdas.
- **Lambda alias canary routing is per-invocation random — not per-user sticky** — the same user can hit different versions on consecutive requests. Both versions must be capable of handling any request independently. Never use Lambda alias routing for features that require session consistency between versions.
- **CloudWatch Lambda metrics require the `Resource` dimension for per-version monitoring** — without it, Errors and Duration metrics are aggregated across all versions behind the alias, making canary health monitoring impossible.

5.6 EKS — Networking (VPC CNI), IRSA, Cluster Autoscaler vs Karpenter

---

🟢 What It Is in Simple Terms

Three critical EKS production concerns: how pods get IP addresses (VPC CNI), how pods securely access AWS services (IRSA), and how the cluster automatically scales nodes up and down (Cluster Autoscaler vs Karpenter).

🧩 VPC CNI — Pod Networking

```
Default Kubernetes CNI assigns pod IPs from an overlay network
(e.g., 10.244.0.0/16) — separate from the VPC.

AWS VPC CNI (aws-node DaemonSet):
Assigns REAL VPC IP addresses directly to pods.

How it works:
1. VPC CNI runs as DaemonSet on every node
2. Allocates secondary IP addresses on node's ENI
3. Each pod gets one of these secondary IPs
4. Pod IPs are routable within the VPC natively

                    Node ENI
┌──────────────────────────────────────────┐
│ Primary IP: 10.0.1.5 (node itself)       │
│ Secondary IPs: 10.0.1.10                 │ → Pod A: 10.0.1.10
│               10.0.1.11                 │ → Pod B: 10.0.1.11
│               10.0.1.12                 │ → Pod C: 10.0.1.12
└──────────────────────────────────────────┘

Benefits:
├── Pods communicate with RDS, ElastiCache, etc. natively (no NAT)
├── Security Groups can be applied to individual pods
└── VPC Flow Logs show pod-level traffic

⚠ ENI limits cap pod density per instance:
m5.large:  3 ENIs × 10 secondary IPs = 29 pods max
m5.xlarge: 4 ENIs × 15 secondary IPs = 58 pods max
(You may need LARGE instances to fit many pods, not for CPU/RAM!)

Solutions:
1. Choose instances with more ENI capacity
2. Enable ENABLE_PREFIX_DELEGATION:
   Assigns /28 prefix (16 IPs) per ENI slot instead of 1 IP
   m5.large with prefix delegation: 3 × 10 × 16 = 464 pods!
3. Use custom networking (pods use different CIDR than nodes)

Security Groups for Pods:
├── Assign a SG directly to specific pods (SecurityGroupPolicy CRD)
├── Replaces per-node SG with per-pod SG
├── Example: only DB pods get SG allowing RDS port 5432
└── Requires: branch ENIs on Nitro instances
```

🧩 IRSA — IAM Roles for Service Accounts

```
Problem: How does a pod securely access AWS services?

❌ Bad: Attach permissions to EC2 node IAM role
   → ALL pods on that node get ALL those permissions
   → One compromised pod = entire node's AWS access exposed

✅ Correct: IRSA (IAM Roles for Service Accounts)
   Each pod gets its own IAM role with minimal permissions.

How IRSA works:
1. Create OIDC provider for your EKS cluster
2. Create IAM role with trust policy scoped to specific
   ServiceAccount in specific namespace
3. Annotate Kubernetes ServiceAccount with role ARN
4. Pod using that ServiceAccount gets temp AWS credentials
   injected automatically as env vars
```

```
# Step 1: Get OIDC issuer URL
aws eks describe-cluster --name prod-cluster \
  --query "cluster.identity.oidc.issuer" --output text

# Step 2: Create OIDC provider in IAM
aws iam create-open-id-connect-provider \
  --url https://oidc.eks.us-east-1.amazonaws.com/id/EXAMPLED539 \
  --client-id-list sts.amazonaws.com
```

```
// Step 3: IAM role trust policy (scoped to specific ServiceAccount)
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {
      "Federated": "arn:aws:iam::123:oidc-provider/oidc.eks..."
    },
    "Action": "sts:AssumeRoleWithWebIdentity",
    "Condition": {
      "StringEquals": {
        "oidc.eks.../id/...:sub":
          "system:serviceaccount:production:s3-reader-sa"
      }
    }
  }]
}
```

```
# Step 4: Annotate Kubernetes ServiceAccount
apiVersion: v1
```

```yaml
kind: ServiceAccount
metadata:
  name: s3-reader-sa
  namespace: production
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::123:role/s3-reader-role
---
# Step 5: Pod uses the ServiceAccount
apiVersion: apps/v1
kind: Deployment
spec:
  template:
    spec:
      serviceAccountName: s3-reader-sa  # Pod automatically gets role
      containers:
      - name: app
        image: my-app
```

```
Under the hood:
├── EKS injects projected volume with OIDC token into pod
├── AWS SDK discovers token via AWS_WEB_IDENTITY_TOKEN_FILE env var
├── SDK calls sts:AssumeRoleWithWebIdentity → gets temp credentials
└── SDK rotates credentials automatically before expiry
```

🧩 Cluster Autoscaler vs Karpenter

```
Cluster Autoscaler (CA):
├── Official Kubernetes project, works with any cloud
├── Watches for Unschedulable pods → triggers ASG scale-out
├── Works within pre-defined ASGs (node groups)
├── Scale-in: waits for pods to be rescheduled, then terminates
└── Slow: ASG scale-out can take 3-5 minutes

How CA works:
Pod Unschedulable (no node has capacity)
→ CA detects Unschedulable condition
→ CA calls ASG to increase desired count by 1
→ ASG launches EC2 (2-3 min to register with cluster)
→ Pod scheduled on new node

Problems with CA:
├── Must pre-define node groups with specific instance types
├── Doesn't automatically choose cheapest/best instance type
├── Can't diversify Spot across many instance types easily
├── Can't bin-pack: adds whole node for one small pod
└── No automatic node consolidation for cost savings

Karpenter:
```

```
├── AWS-created, EKS-native (open source)
├── Directly calls EC2 APIs (no ASG required)
├── Provisions right-sized instance for the pending pods
├── Diversifies Spot across many instance types automatically
├── Faster: 45-60 seconds from unschedulable to running
└── Consolidation: bins underutilized pods onto fewer nodes
```

```yaml
# Karpenter NodePool definition
apiVersion: karpenter.sh/v1beta1
kind: NodePool
metadata:
  name: default
spec:
  template:
    spec:
      requirements:
        - key: kubernetes.io/arch
          operator: In
          values: ["amd64", "arm64"]
        - key: karpenter.sh/capacity-type
          operator: In
          values: ["spot", "on-demand"]
        - key: karpenter.k8s.aws/instance-family
          operator: In
          values: ["m5", "m6i", "m6a", "m7g", "c5", "c6i"]
      nodeClassRef:
        apiVersion: karpenter.k8s.aws/v1beta1
        kind: EC2NodeClass
        name: default
  disruption:
    consolidationPolicy: WhenUnderutilized
    consolidateAfter: 30s
```

```
Karpenter consolidation (key differentiator):
If 3 nodes each 20% utilized → consolidate to 1 node.
Karpenter drains and terminates idle nodes automatically.
CA cannot do this efficiently.

CA vs Karpenter decision:
┌──────────────────┬──────────────────┬──────────────────────┐
│ Feature          │ Cluster Autoscaler │ Karpenter            │
├──────────────────┼──────────────────┼──────────────────────┤
│ Spot diversification│ Limited          │ Excellent            │
│ Speed            │ 3-5 minutes      │ 45-60 seconds        │
│ Cost optimization │ Moderate         │ Excellent (consolidation│
│ Multi-arch (ARM) │ Separate node group │ Automatic            │
│ Multi-cloud      │ ✅ Works anywhere  │ ❌ AWS-specific      │
└──────────────────┴──────────────────┴──────────────────────┘
```

```
Use Karpenter for new EKS clusters on AWS.
Use CA only if you need multi-cloud portability.
```

💬 Short Crisp Interview Answer

"Three critical EKS production topics. VPC CNI assigns real VPC IPs to pods — pods are natively routable within the VPC but ENI secondary IP limits cap pod density per node. Enable prefix delegation to dramatically increase pod capacity. IRSA solves the problem of pods accessing AWS services — instead of permissive node IAM roles, you create per-pod IAM roles federated via the cluster's OIDC provider and annotate Kubernetes ServiceAccounts with the role ARN. Each pod gets scoped temporary credentials automatically. For scaling, Karpenter is the modern choice — it directly provisions right-sized instances in 45 seconds, diversifies Spot across many instance types, and consolidates underutilized nodes. The only reason to use Cluster Autoscaler today is multi-cloud portability."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
ENI limit = pod limit	Max pods = (ENIs-1) × secondary IPs + 1. Enable prefix delegation for high pod density
IRSA OIDC per cluster	Each EKS cluster needs its own OIDC provider registered in IAM
aws-auth for Karpenter nodes	Karpenter-provisioned nodes must be in aws-auth (or use access entries API)
Karpenter consolidation disruption	Can evict pods to consolidate. Use PodDisruptionBudgets to protect critical workloads
CA with Spot	CA doesn't handle Spot interruptions gracefully without extra tooling
IRSA token rotation	AWS SDK handles rotation automatically. Custom code must handle token refresh

- **IRSA OIDC provider is per-cluster** — each EKS cluster needs its own OIDC provider registered in IAM. If you have 5 clusters, you need 5 OIDC providers.
- **Node IAM role should have zero application permissions** — it should only contain permissions needed for the node itself to function (ECR pulls, CloudWatch logs, VPC CNI). All application AWS access goes through IRSA.

5.7 Lambda — Cold Starts, Provisioned Concurrency, SnapStart

🟢 What It Is in Simple Terms

Cold starts are the latency penalty when Lambda initializes a new execution environment. Provisioned Concurrency pre-warms environments to eliminate that penalty. SnapStart (Java) takes a snapshot after initialization so the JVM startup cost is paid only once.

⚙️ Cold Start Deep Dive

```
Cold start timeline:

   0ms                                          300ms+
   │──────────────────────────────────────────────────│
   │ AWS allocates  │ Download   │ Init    │ Handler  │
   │ microVM        │ code/image │ code    │ runs     │
   │ (Firecracker)  │ (from S3)  │ runs    │          │
   │──────────────────────────────────────────────────│
   └── ALL of this is "cold start latency" ────────────┘

Typical cold start durations:
├── Python:  ~100-300ms
├── Node.js: ~100-300ms
├── Go:      ~100-200ms (compiled, minimal init)
├── .NET:    ~500ms-1s
└── Java:    ~1-10 SECONDS (JVM + class loading + Spring etc.)

What causes cold starts:
├── First invocation of a function
├── Scaling beyond current warm environments (traffic spike)
├── New deployment (all environments recycled)
└── After ~15 minutes of inactivity (environments recycled)

What DOESN'T cause cold starts:
└── Subsequent invocations on a warm (reused) environment

Reducing cold start duration:
├── Minimize deployment package size (faster download)
├── Move heavy initialization OUTSIDE the handler
│   (DB connections, SDK clients — reuse across invocations)
├── Choose lightweight runtimes (Python/Node over Java)
├── Use arm64 (Graviton2) — faster init + 20% lower cost
└── Lazy-load what you can
```

```python
import boto3

# ✅ OUTSIDE handler = runs once per cold start (reused across
invocations)
s3 = boto3.client('s3')
dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('orders')

def handler(event, context):
```

```
    # ✅ Clients already initialized — no re-initialization on warm
invocations
    response = table.get_item(Key={'id': event['id']})
    return response['Item']

# ❌ INSIDE handler = re-initializes on EVERY invocation (expensive!)
def bad_handler(event, context):
    s3 = boto3.client('s3')          # re-created every time
    dynamodb = boto3.resource('dynamodb')  # re-created every time
    table = dynamodb.Table('orders')
    return table.get_item(Key={'id': event['id']})['Item']
```

🧩 Provisioned Concurrency

```
What it does:
Pre-initializes a specific number of execution environments.
Those environments are ALWAYS warm → zero cold start for those slots.

How it works:
├── Lambda initializes N environments ahead of time
├── Init code runs during provisioning (not on first invocation)
├── Invocations hitting provisioned capacity = no cold start
├── Invocations beyond provisioned capacity = cold start (new env)
└── AWS keeps provisioned envs warm indefinitely (while configured)

Cost:
├── Provisioned Concurrency: $0.015 per GB-hour (while provisioned)
├── Plus normal compute cost when invoked
└── Example: 512MB function, 10 provisioned, 24/7:
    10 × 0.5GB × 24hr × 30days × $0.015 = $54/month overhead

When NOT to use Provisioned Concurrency
Only enable Provisioned Concurrency for: (1) customer-facing APIs with
measurable p99 cold start impact, (2) functions invoked frequently enough
that warm environments are actually used. Do NOT enable for: batch jobs,
event-driven async functions, functions invoked less than once per minute
— cold start cost is negligible compared to PC overhead.
```

```
# Step 1: Publish a version (PC applies to versions, not $LATEST)
aws lambda publish-version --function-name my-api

# Step 2: Create alias pointing to the version (recommended pattern)
aws lambda create-alias \
  --function-name my-api \
  --name prod \
  --function-version 5

# Step 3: Set Provisioned Concurrency on the alias
```

```
aws lambda put-provisioned-concurrency-config \
  --function-name my-api \
  --qualifier prod \
  --provisioned-concurrent-executions 50

# Step 4: Auto-scale Provisioned Concurrency with utilization tracking
aws application-autoscaling put-scaling-policy \
  --policy-name pc-utilization \
  --service-namespace lambda \
  --resource-id function:my-api:prod \
  --scalable-dimension lambda:function:ProvisionedConcurrency \
  --policy-type TargetTrackingScaling \
  --target-tracking-scaling-policy-configuration '{
    "PredefinedMetricSpecification": {
      "PredefinedMetricType": "LambdaProvisionedConcurrencyUtilization"
    },
    "TargetValue": 0.7
  }'
# Scales up PC when 70% of provisioned environments are in use
```

🧩 Lambda SnapStart (Java)

```
Problem: Java Lambda cold starts take 1-10+ seconds due to:
├── JVM startup time
├── Class loading
└── Application framework init (Spring, Quarkus, etc.)

SnapStart solution:
1. Lambda initializes the function execution environment
2. Takes a snapshot of memory and disk state AFTER init completes
3. Snapshot stored encrypted in S3
4. On each cold start: RESTORE from snapshot
   → Snapshot restore = ~100-200ms instead of 1-10 seconds

Supported runtimes:
└── Java 11 and Java 17 (Amazon Corretto)

SnapStart vs Provisioned Concurrency cost comparison
SnapStart has no additional cost beyond normal Lambda invocation pricing.
Provisioned Concurrency charges per GB-hour regardless of invocations. For
Java functions, SnapStart is almost always the right first choice — try it
before spending on Provisioned Concurrency.
```

```
# Enable SnapStart on function creation
aws lambda create-function \
  --function-name java-api \
  --runtime java17 \
  --snap-start ApplyOn=PublishedVersions \
```

```
  --handler com.example.Handler::handleRequest \
  --role arn:aws:iam::123:role/lambda-role \
  --zip-file fileb://function.zip
```

```
// SnapStart lifecycle hooks — handle stateful init correctly
import org.crac.Core;
import org.crac.Resource;

public class Handler implements Resource {
    private Connection dbConnection;

    public Handler() {
        Core.getGlobalContext().register(this);
        dbConnection = DriverManager.getConnection(DB_URL);
    }

    @Override
    public void beforeCheckpoint(Context<? extends Resource> context) {
        // Called BEFORE snapshot is taken
        // Close connections — they won't survive snapshot restore
        dbConnection.close();
    }

    @Override
    public void afterRestore(Context<? extends Resource> context) {
        // Called AFTER restore from snapshot
        // Re-open connections for the new invocation environment
        dbConnection = DriverManager.getConnection(DB_URL);
    }
}
```

```
⚠ SnapStart critical gotchas:
├── Only on published versions (not $LATEST)
├── Snapshot taken ONCE per deployment — frozen state
├── Random UUIDs, timestamps, random seeds in init code
│   are FROZEN in snapshot and REPLAYED every invocation
│   → Same UUID returned on every cold start!
├── Network connections in init DON'T survive snapshot restore
│   → Must re-establish in afterRestore lifecycle hook
└── Use CRaC lifecycle hooks (beforeCheckpoint / afterRestore)
    for any stateful initialization
```

💬 Short Crisp Interview Answer

"Cold starts are the latency cost of Lambda initializing a new execution environment — downloading code, starting the runtime, and running init code. Duration ranges from 100ms for Python/Node to multiple

seconds for Java with Spring. Mitigation strategies include minimizing deployment package size, moving client initialization outside the handler for reuse, and choosing lighter runtimes. Provisioned Concurrency pre-warms N environments so those invocations have zero cold start — you pay for them whether invoked or not, so pair with auto-scaling at 70% utilization. SnapStart is specifically for Java — Lambda snapshots the fully-initialized environment and restores from it on cold start, reducing Java latency to ~100ms. The critical SnapStart gotcha: random state or timestamps initialized in the constructor are frozen in the snapshot and replayed for every invocation — use CRaC lifecycle hooks to re-initialize state after restore."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
Init code runs on cold start only	Not every invocation — only on env initialization. Confusing for newcomers
Provisioned Concurrency not on $LATEST	Must publish a version first. PC only works on versions and aliases
SnapStart frozen state	UUIDs, timestamps, random numbers in init code are frozen → same value replayed every invocation
SnapStart network connections	Don't survive snapshot. Must re-connect in afterRestore hook
Auto-scaling PC lag	Scaling up PC takes 2-3 minutes. Use aggressive scale-out, conservative scale-in
Provisioned vs Reserved	PC instances count toward reserved concurrency. Set reserved high enough to include all PC

- SnapStart vs Provisioned Concurrency cost comparison — SnapStart has zero additional cost. Provisioned Concurrency charges per GB-hour regardless of invocations. For Java functions, always try SnapStart first before committing to Provisioned Concurrency spend.
- Provisioned Concurrency auto-scaling lag — scaling up PC takes 2-3 minutes. Configure aggressive scale-out, conservative scale-in to avoid cold starts during rapid traffic ramps.

5.8 App Mesh & Service Mesh Concepts

🟢 What It Is in Simple Terms

A service mesh is a dedicated infrastructure layer for service-to-service communication. Instead of your application code handling retries, timeouts, circuit breaking, and mTLS, a sidecar proxy does it transparently. AWS App Mesh is AWS's managed service mesh using Envoy as the proxy.

🔍 Why Service Meshes Exist

```
In microservices, 50+ services talk to each other.
Every service needs:
├── Retries (with exponential backoff)
├── Timeouts
├── Circuit breaking (stop calling failing services)
├── mTLS (service-to-service encryption + identity)
├── Load balancing (client-side)
├── Traffic shifting (canary deployments)
├── Observability (distributed tracing, per-route metrics)
└── Rate limiting

Option 1: Build this into every service
→ Each team implements it in Go, Python, Java, Node...
→ Inconsistent across teams and languages
→ Hard to enforce org-wide policies

Option 2: Service Mesh
→ Sidecar proxy intercepts all traffic (no app code changes)
→ All policies configured centrally in control plane
→ Language-agnostic — proxy handles it, not the app
→ Uniform observability across all services automatically
```

⚙️ How App Mesh Works

```
App Mesh Architecture:

┌──────────────────────────────────────────────────────────┐
│  App Mesh Control Plane (AWS managed)                    │
│  Mesh config, routing rules, retry policies, mTLS        │
└──────────────────────────────────────────────────────────┘
                        │
                        │ xDS protocol (pushes config to Envoy)
                        ▼
Data Plane (your VPC):

┌──────────────────────┐          ┌──────────────────────┐
│  Service A Pod/Task  │          │  Service B Pod/Task  │
│  ┌────────────────┐  │          │  ┌────────────────┐  │
│  │ Your App       │  │          │  │ Your App       │  │
│  │ (port 8080)    │  │          │  │ (port 8080)    │  │
│  └────────┬───────┘  │   mTLS   │  └────────┬───────┘  │
│           ▼          │  ◄──────►│           ▼          │
│  ┌────────────────┐  │          │  ┌────────────────┐  │
│  │ Envoy Sidecar  │  │          │  │ Envoy Sidecar  │  │
│  │ (proxy)        │  │          │  │ (proxy)        │  │
│  └────────────────┘  │          │  └────────────────┘  │
└──────────────────────┘          └──────────────────────┘

App code calls http://service-b:8080
→ Envoy intercepts → applies retries, mTLS, circuit breaking
→ Your app code changes nothing
```

🧩 App Mesh Components

```
# Create the mesh
aws appmesh create-mesh --mesh-name prod-mesh

# Create Virtual Node (represents a deployment)
aws appmesh create-virtual-node \
  --mesh-name prod-mesh \
  --virtual-node-name product-v1 \
  --spec '{
    "serviceDiscovery": {
      "awsCloudMap": {
        "namespaceName": "prod.local",
        "serviceName": "product"
      }
    },
    "listeners": [{"portMapping": {"port": 8080, "protocol": "http"}}],
    "backends": [{
      "virtualService": {"virtualServiceName": "database.prod.local"}
    }]
  }'

# Create Route with canary traffic split + retry policy
aws appmesh create-route \
  --mesh-name prod-mesh \
  --virtual-router-name product-router \
  --route-name canary-route \
  --spec '{
    "httpRoute": {
      "match": {"prefix": "/"},
      "action": {
        "weightedTargets": [
          {"virtualNode": "product-v1", "weight": 90},
          {"virtualNode": "product-v2", "weight": 10}
        ]
      },
      "retryPolicy": {
        "perRetryTimeout": {"value": 2, "unit": "s"},
        "maxRetries": 3,
        "httpRetryEvents": ["server-error", "gateway-error"]
      }
    }
  }'
```

🧩 App Mesh vs Istio

```
┌──────────────────────┬──────────────────┬──────────────────────┐
│ Feature              │ App Mesh         │ Istio on EKS         │
├──────────────────────┼──────────────────┼──────────────────────┤
│ Control plane        │ AWS managed      │ Self-managed         │
│ Proxy                │ Envoy            │ Envoy                │
│ mTLS                 │ ✅ (via ACM PCA) │ ✅                   │
│ Traffic shifting     │ ✅               │ ✅                   │
│ Circuit breaking     │ ✅               │ ✅                   │
│ Observability        │ CloudWatch+X-Ray │ Prometheus+Jaeger    │
│ AWS integration      │ Deep native      │ Manual config        │
│ Feature richness     │ Medium           │ Very high            │
│ Operational complexity │ Low            │ High                 │
│ Works with ECS       │ ✅               │ ❌ (EKS only)        │
└──────────────────────┴──────────────────┴──────────────────────┘

App Mesh: good for AWS-native environments, works with ECS and EKS.
Istio: richer features, more control, more complexity, EKS only.

Note: AWS is increasingly recommending Istio over App Mesh for EKS.
App Mesh is being positioned as legacy for EKS workloads.
For new EKS deployments, consider Istio or network policies + AWS LBC.
```

💬 Short Crisp Interview Answer

"A service mesh is an infrastructure layer for service-to-service communication — instead of each microservice implementing retries, circuit breaking, mTLS, and observability, a sidecar proxy (Envoy) handles it transparently at the network level without code changes. AWS App Mesh is AWS's managed service mesh — you define Virtual Nodes (deployments), Virtual Services (logical names), and Routes (traffic rules including weighted splits for canary). App Mesh works with both ECS and EKS which is a key advantage over Istio. The main benefit is language-agnostic, centrally configured traffic policy and uniform observability across all services."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
App Mesh only controls mesh traffic	Traffic to RDS, S3, external services is not managed by App Mesh
Sidecar injection	In EKS: configure mutating webhook. In ECS: add envoy container to task definition
mTLS requires ACM Private CA	Extra cost: ~$400/month per private CA
App Mesh deprecation signal	AWS recommending Istio on EKS for new deployments

---

5.9 Fargate — Resource Limits, Networking, Security

🟢 What It Is in Simple Terms

A deep dive into Fargate's specific constraints, networking model, and security model — understanding the details that trip people up in production.

🧩 Resource Limits

```
Fargate task CPU and memory — VALID combinations only:

CPU value      Memory range
──────────     ────────────
0.25 vCPU  →  0.5GB to 2GB   (0.5GB increments)
0.5 vCPU   →  1GB to 4GB     (1GB increments)
1 vCPU     →  2GB to 8GB     (1GB increments)
2 vCPU     →  4GB to 16GB    (1GB increments)
4 vCPU     →  8GB to 30GB    (1GB increments)
8 vCPU     →  16GB to 60GB   (4GB increments)
16 vCPU    →  32GB to 120GB  (8GB increments)

⚠ Arbitrary combinations are REJECTED.
   0.25 vCPU with 8GB → INVALID → task fails to register.
   Must match the table above.

Container vs Task resource allocation:
Task-level CPU/memory = total budget for ALL containers in the task
Container-level = allocation within the task budget

⚠ If sum of container hard limits > task allocation → task fails.
   Container memory = hard limit (OOM killed if exceeded).
   memoryReservation = soft limit (can burst if task has spare).

Storage (ephemeral):
├── 20GB ephemeral storage by default (included in price)
├── Up to 200GB: set ephemeralStorage.sizeInGiB in task definition
├── Shared across all containers in the task
└── DELETED when task stops — ephemeral!

Persistence on Fargate:
├── EFS: works with Fargate ✅  (persistent, shared)
└── EBS: does NOT work with Fargate ❌  (EC2 only)
```

🧩 Fargate Networking

```
Fargate = awsvpc mode ONLY.
No bridge. No host. Only awsvpc.

Each task gets:
├── Its own ENI (Elastic Network Interface)
├── Its own private IP (from subnet CIDR)
├── Its own Security Group(s)
└── Optionally: public IP (assignPublicIp=ENABLED)

⚠ assignPublicIp=ENABLED:
   Task gets public IP → can reach internet without NAT Gateway
   But also potentially reachable FROM the internet!
   Only use if Security Group is correctly locked down.
   PREFER: private subnet + NAT Gateway for production backends.

Traffic routing from Fargate tasks:

Private subnet (DISABLED):
├── Internet access:  via NAT Gateway in public subnet
├── AWS services:     via VPC Endpoints (free for S3/DynamoDB)
└── Internal VPC:     direct routing (no NAT needed)

Public subnet (ENABLED):
├── Internet access:  via public IP directly
├── Inbound:          protected only by Security Group
└── Less secure — avoid for production application backends

Fargate + ALB:
├── ALB target type MUST be "ip" (not "instance")
│   Fargate tasks register by ENI IP, not EC2 instance ID
├── ALB and Fargate tasks can be in different subnets
│   (ALB in public subnet, Fargate in private subnet)
└── ALB SG must allow outbound to Fargate task SG on container port

Container-to-container within same task:
├── All containers share same ENI and network namespace
├── Talk to each other via localhost:containerPort
└── Cannot assign different Security Groups per container
    (they share the same ENI)
```

🧩 Fargate Security

```
Isolation model:
├── Each task runs in its own Firecracker microVM
├── Kernel isolation from other customers' tasks
├── No shared kernel between Fargate tasks
└── Stronger isolation than containers on shared EC2

IAM Roles:
```

```
  ┌──────────────────────────────────────────────────────────┐
  │ Task Execution Role:                                      │
  │ - Pull image from ECR                                     │
  │ - Write logs to CloudWatch                                │
  │ - Read secrets from Secrets Manager / Parameter Store     │
  │ Used by: ECS Agent (infrastructure layer)                 │
  ├──────────────────────────────────────────────────────────┤
  │ Task Role:                                                │
  │ - Call S3, DynamoDB, SQS, etc.                            │
  │ Used by: your application code inside containers          │
  └──────────────────────────────────────────────────────────┘

Secrets injection:
├── Via Secrets Manager: "valueFrom": "arn:...secret"
├── Via Parameter Store: "valueFrom": "arn:...param"
├── Injected as environment variables at task startup
└── ⚠ Env var secrets visible via DescribeTasks API
        Consider reading secrets at runtime in code instead.

Linux capabilities on Fargate:
└── ALL capabilities dropped by default.
    Cannot add: NET_ADMIN, SYS_PTRACE, or other privileged caps.
    Cannot run privileged containers on Fargate.
```

```yaml
# Hardened Fargate task definition
containerDefinitions:
  - name: app
    image: 123.dkr.ecr.us-east-1.amazonaws.com/app@sha256:abc123
    readonlyRootFilesystem: true    # can't write to root FS
    user: "1000:1000"               # run as non-root
    mountPoints:
      - containerPath: /tmp         # explicitly allow /tmp
        sourceVolume: tmp-vol
    logConfiguration:
      logDriver: awslogs
      options:
        awslogs-group: /ecs/prod/app
        awslogs-stream-prefix: ecs
```

💬 Short Crisp Interview Answer

"Fargate has specific resource combinations you must follow — CPU and memory are tied to valid pairings, and the sum of container allocations can't exceed the task budget. Networking is awsvpc only — each task gets its own ENI, private IP, and security group. For internet access from private subnets, traffic goes through a NAT Gateway. Always keep Fargate tasks in private subnets. Security-wise, Fargate provides strong kernel-level isolation via Firecracker microVMs. The two IAM roles are critical — execution role for ECS infrastructure (ECR pulls, CloudWatch logs, Secrets Manager), task role for your application's AWS API

calls. For production: pin images to ECR digest not tag, enable read-only root filesystem, run as non-root user, and use ECS Exec with CloudWatch logging for auditable container access."

🏭 Real World Production Example

```
Production Fargate hardening checklist:

Task Definition:
├── image: 123.dkr.ecr.amazonaws.com/app@sha256:abc123 (pinned digest)
├── readonlyRootFilesystem: true
├── user: "1000:1000" (non-root)
├── logConfiguration: awslogs → /ecs/prod/app
├── secrets: [DB_PASSWORD from Secrets Manager]
└── healthCheck: curl -f /health || exit 1

Network:
├── subnets: [private-subnet-a, private-subnet-b]
├── assignPublicIp: DISABLED
└── securityGroups: inbound port 8080 from ALB SG only

IAM:
├── executionRole: ECR pull + CW logs + Secrets Manager read
└── taskRole: specific S3 bucket + specific DynamoDB table only

Service:
├── deploymentType: Blue/Green via CodeDeploy
├── rollback: automatic on health check failure
└── enableExecuteCommand: true (audited via CloudWatch)

Monitoring:
├── Container Insights: CPU/memory per task
├── CloudWatch alarms: task count, CPU > 80%
└── X-Ray sidecar: distributed tracing
```

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
Invalid CPU/memory combo	Arbitrary combinations rejected. Must use valid pairs
ALB target type must be "ip"	Not "instance" — Fargate tasks register by IP, not EC2 instance ID
ENI limits per subnet	Subnet must have enough available IPs for all tasks
Containers share ENI in same task	Can't assign different SGs per container — they share the ENI
Privileged containers blocked	Fargate drops all Linux capabilities. No NET_ADMIN, no SYS_PTRACE
No persistent EBS	Fargate ephemeral storage only. Use EFS for persistence

Gotcha	Detail
Secrets as env vars	Visible via DescribeTasks API. For highly sensitive data, read in code at runtime

---

🔗 Category 5 — Full Connections Map

```
CONTAINERS & SERVERLESS connects to:

ECS
├── ECR           → image registry (pull at task launch)
├── ALB/NLB       → load balancer for services (target type: ip)
├── IAM           → task role (app) + execution role (ECS infra)
├── CloudWatch    → container logs, Container Insights metrics
├── Secrets Mgr   → inject secrets via execution role
├── EFS           → persistent shared storage for tasks
├── SQS           → worker tasks poll queues (trigger scaling)
├── EventBridge   → trigger ECS run-task for scheduled jobs
├── CodeDeploy    → blue/green deployments for ECS services
├── App Mesh      → service mesh via Envoy sidecar
├── Cloud Map     → service discovery between ECS services
└── X-Ray         → distributed tracing via daemon sidecar

EKS
├── ECR           → image registry for pods
├── ALB/NLB       → via AWS Load Balancer Controller (ingress)
├── IAM (IRSA)    → per-pod IAM roles via OIDC federation
├── VPC CNI       → pod IP addressing from VPC subnet CIDR
├── EFS/EBS CSI   → persistent volumes for stateful workloads
├── CloudWatch    → Container Insights, Fluent Bit log forwarding
├── Karpenter     → node auto-provisioning (direct EC2 API)
├── ACM           → TLS certificates for ALB ingress
├── Secrets Store → External Secrets Operator or ASCP
└── App Mesh/Istio → service mesh for inter-pod communication

Lambda
├── API Gateway   → HTTP trigger (synchronous invocation)
├── ALB           → HTTP trigger (synchronous invocation)
├── S3            → event trigger (asynchronous)
├── SQS           → polling trigger (Event Source Mapping)
├── SNS           → push trigger (asynchronous)
├── EventBridge   → scheduled + custom event trigger
├── DynamoDB Streams → change data capture trigger
├── Kinesis       → stream processing trigger
├── CloudWatch    → monitoring, logs (/aws/lambda/fn-name)
├── X-Ray         → distributed tracing (active tracing)
├── IAM           → function execution role
├── VPC           → Lambda can run inside VPC (uses ENI)
├── EFS           → persistent storage for Lambda functions
```

```
  └── Layers        → shared dependencies and runtimes

ECR
  ├── ECS           → image pull at task start
  ├── EKS           → image pull at pod start
  ├── CodeBuild     → push images after CI build
  ├── AWS Inspector → enhanced vulnerability scanning
  └── IAM           → repository policies for cross-account access
```

📌 Quick Reference — Interview Cheat Sheet

Topic	Key Fact
ECS Task Role	What your app code can call (S3, DDB, SQS)
ECS Execution Role	What ECS infra can do (ECR pull, CW logs, Secrets Mgr)
ECS service deployment types	Rolling Update (default), Blue/Green (CodeDeploy), External
ECS essential=true	If essential container exits → whole task stops immediately
ECS Exec requirement	enableExecuteCommand=true must be set at service creation
ECR lifecycle policy	Required to prevent unbounded image storage cost
ECR tag immutability	Prevents overwriting existing production image tags
ECR auth token expiry	12 hours. ECS auto-refreshes. CI pipelines must refresh manually
Fargate networking	awsvpc mode only. Each task = own ENI + own Security Group
Fargate max resources	16 vCPU, 120 GB memory per task
Fargate CPU/memory	Must use valid pairs from the spec table
Fargate vs EC2	Fargate = no server mgmt. EC2 = GPU, custom AMI, cheaper at scale
Fargate Spot savings	Up to 70% cheaper, 2-min interruption warning
Fargate isolation	Firecracker microVM per task. Stronger than shared-EC2 containers
EKS control plane cost	$0.10/hr = $72/month flat regardless of node count
EKS MNG vs self-managed	MNG = graceful drain + AMI updates + Spot. Self = custom AMI
EKS Fargate DaemonSet	NOT supported. Use sidecar containers instead
VPC CNI pod IP limit	Max pods = (ENIs × secondary IPs) - 1. Enable prefix delegation
IRSA mechanism	Per-pod IAM via OIDC. Never put app permissions on node role
Karpenter vs CA speed	Karpenter: 45-60s direct EC2. Cluster Autoscaler: 3-5min via ASG
Karpenter vs CA Spot	Karpenter diversifies across many types. CA limited to defined node groups

Topic	Key Fact
Karpenter consolidation	Bins underutilized pods onto fewer nodes, terminates idle nodes
Lambda cold start worst	Java (JVM + Spring): 1-10 seconds. Python/Node: ~100-300ms
Lambda init code location	Outside handler = runs once per cold start. Inside = every invocation
Lambda concurrency types	Unreserved (shared pool), Reserved (guarantee + cap), Provisioned (pre-warmed)
Lambda concurrency formula	concurrent = req/sec × avg_duration_sec
Lambda SnapStart	Java 11/17 only. Snapshot post-init state → ~100ms cold start
SnapStart frozen state	UUIDs/timestamps in init are frozen. Use CRaC hooks to re-init
Lambda SQS visibility timeout	Must be 6× Lambda timeout or messages processed twice
Lambda async retries	2 automatic retries = function runs up to 3 times. Must be idempotent
Lambda layers limit	Up to 5 per function. Total 250MB unzipped. Not auto-updated
Service mesh benefit	Language-agnostic, centrally configured, uniform observability
App Mesh proxy	Envoy sidecar intercepts all traffic. Works with ECS and EKS
Fargate no EBS	Use EFS for persistence. Ephemeral storage deleted on task stop
ALB with Fargate target type	Must be "ip" not "instance"

Category 5: Containers & Serverless — Complete Interview Guide
Excluded topics: CodeBuild, CodePipeline, CloudFormation